

IoT Event Processing Logic

Reference document for tasks.py — fetch, process & store pipeline

Overview

A background worker (iot_data_worker.py) runs continuously, calling fetch_iot_data_job() every 10 seconds. This function fetches new device events from an external API, decodes and validates each event, converts raw sensor values into meaningful readings, and bulk-saves everything to the database. If any error reading is High or Critical priority, notifications are triggered automatically. This document lists every condition and formula used in that logic.

1. Fetching & Initial Filtering

API call: **GET** <https://mdata.air2o.net/events/>

- If status code is not 200 → stop, log failure, exit.
- If the response is empty → stop, log "no events", exit.
- Events are sorted ascending by event_id, so the oldest event is processed first.

2. Duplicate Prevention

Two separate lookup sets are built before processing begins:

```
existing_ids          = event_ids already saved in SensorReading table
existing_error_ids    = event_ids already saved in ErrorCodeReading table
```

If an event's ID is already in existing_ids, it is skipped immediately with reason "Already exists".

3. Per-Event Validation Conditions (checked in order)

#	Condition	Result if it fails
1	event_id missing OR payload missing/empty	Skip — "Missing event_id or payload"
2	event_id already exists in the database	Skip — "Already exists"
3	device_id missing OR batch_type missing after parsing	Skip — "Missing device_id or batch_type"
4	device_id not found in ServiceItems table	Skip — "No ServiceItem found with pcb_serial_number {device_id}"
5	No valid sensor codes exist for that batch_type	Skip — "No valid sensors for {batch_type}"
6	Zero readings actually created for the event	Skip — "No valid readings"

Timestamp fallback condition:

```
try:
    original_timestamp =.strptime(created_at, "%d-%m-%Y %H:%M:%S")
except ValueError:
    original_timestamp = datetime.utcnow()    # fallback if format doesn't match
```

4. Payload Parsing Logic

The raw payload is comma-separated. Each part is checked against these conditions, in this exact order:

```
if part starts with "DI:"      &rarr; this is the device_id
elif part starts with "0xA1"   &rarr; batch_type = 'Batch 1'
elif part starts with "0xA2"   &rarr; batch_type = 'Batch 2'
elif part starts with "0xA3"   &rarr; batch_type = 'Batch 3'
elif ":" in part AND part does NOT start with "0x"
                                &rarr; treated as a (sensor_code, value) data point
```

Anything that matches none of these is silently ignored.

5. Sensor Code Validity Check

```
if code not in valid_codes and not (batch_type == 'Batch 3' and code == 'DS'):
    continue # skip this data point entirely
```

Plain English: a code is processed only if it belongs to the valid list for that batch type — with one exception: the DS code is always allowed for Batch 3, since it is handled separately (see Section 7).

6. Scaling Formula (Unit Conversion)

```
SCALING_RULES = {
    "ODT": 10, "INDT": 10, "AST": 10, "RT": 10,
    "INPC": 10, "DPC": 10, "P1C": 10, "P2C": 10, "P3C": 10,
    "P1KW": 100, "P2KW": 100, "P3KW": 100, "TP": 100
}
```

Formula applied when a code is in this table:

```
final_value = float(raw_value) / SCALING_RULES[code]
```

Example: ODT: 250 → 250 / 10 → **25.0**

If the code is NOT in the scaling table:

```
if value.isdigit():
    final_value = value # keep as-is, whole number string
else:
    float_val = float(value)
    final_value = str(int(float_val)) if float_val.is_integer() else str(float_val)
    # e.g. "12.0" -> "12", but "12.5" stays "12.5"
```

If conversion throws a ValueError (value isn't a valid number), that single data point is skipped, but the rest of the event still processes normally.

7. Device Status (DS) Bit-Decoding — Batch 3 Only

This is the most complex formula in the file. DS is a single integer where each bit position represents a separate on/off flag:

```
DS_FLAGS = {  
    "EOF": 0, "HORB": 1, "HPHF": 2, "CDF": 3, "HPC": 4, "HPS": 5, "ISOC": 6  
}
```

Bit-extraction formula:

```
flag_value = (device_status >> bit_pos) & 1
```

Plain English: shift the number right by bit_pos places, then keep only the last bit (0 or 1).

Example: if device_status = 13 (binary 1101):

Flag	Bit position	Calculation	Result
EOF	0	13 >> 0 & 1	1
HORB	1	13 >> 1 & 1	0
HPHF	2	13 >> 2 & 1	1
CDF	3	13 >> 3 & 1	1

Each of the 7 flags becomes its own separate SensorReading row, only if that flag code exists in the sensor map — otherwise it is skipped silently.

8. Error Code (EC) Handling — nested inside DS block, Batch 3 only

```
1. Find "EC" among the data_points (if not present -> nothing happens)  
2. Try converting EC value to int -> if it fails: print "Invalid EC value format",  
   skip  
3. if ec_code_int == 0:  
    -> no error, just log "no actionable error"  
4. if ec_code_int != 0:  
    -> this is a real error  
    -> check: is event_id already in existing_error_ids?  
        -> if yes: silently skipped (no new record created)  
        -> if no: look up ec_code_int in error_code_map  
            -> if found: create ErrorCodeReading record  
            -> if not found: print "Error code {ec_code_int} not found in E  
rrorCode table"
```

Note: the existing_error_ids duplicate check does not add an explicit "skipped" log entry or count — it just silently avoids creating a duplicate record. This may be worth tightening later, but that is the current behavior.

9. Success / Failure Bookkeeping Per Event

```
inserted_for_event = True    # set whenever ANY reading (sensor or DS flag) is added
```

- If **true** at the end of the event loop → `inserted_count += 1`, and `batch_inserted[batch_type] += 1`
- If **false** → `skipped_count += 1`, `batch_skipped[batch_type] += 1`, logged as "No valid readings"

10. Bulk Save (Database Transaction)

```
with transaction.atomic():  
    SensorReading.objects.bulk_create(..., batch_size=500)  
    ErrorCodeReading.objects.bulk_create(..., batch_size=500)
```

Plain English: both saves happen together as one all-or-nothing database transaction — if anything fails mid-way, nothing is partially saved.

11. Notification Trigger Condition

```
if error.priority in ['HIGH', 'CRITICAL']:  
    -&gt; notify Service Managers of that company  
    -&gt; notify the Customer who owns the device  
    -&gt; notify Delegates where can_monitor_equipment = True AND status = 'Active'
```

Each notification only fires if the recipient has an `fc_m_token` set on their record — otherwise it is silently skipped for that person.

This document reflects the logic exactly as implemented in `tasks.py` at the time of writing. Any future changes to scaling rules, batch types, or validation conditions should be reflected here as well.